IAM LAB REPORT (LABS 1–12)

Course Title: Identity and Access Management
Environment: AlmaLinux 9, CentOS 7, Ubuntu 22.04 Virtual Machines

Course Title: Bochu Yunwen Eric
Environment: UBa25EP115

Lab 1: Assigning Limited Sudo Privileges **5**
Objective 5
Background 5
Procedure 5
Results and Observations 6
Analysis 6
Conclusion 6
Lab 2: Disabling the Sudo Timer **6**
Objective 6
Background 6
Procedure 6
Results 8
Analysis 8
Conclusion 8
Lab 3: Encrypted Home Directory **8**
Objective 8
Background 8
Procedure 8
Results 9
Analysis 9
Conclusion 9
Lab 4: Password Complexity Criteria **9**
Objective 9
Background 9
Procedure 9
Results 11
Analysis 11
Conclusion 11
Lab 5: Account and Password Expiry **11**
Objective 11
Background 11
Procedure 11
Results 12
Analysis 12
Conclusion 12
Lab 6: Detecting Compromised Passwords **13**
Objective 13
Background 13
Procedure 13
Results 14
Analysis 14
Conclusion 14
Lab 7: Configuring pam_tally2 ..... **15**

Objective..... 15
Background..... 15
Procedure..... 15
Results..... 16
Analysis..... 16
Conclusion..... 16
Lab 8: Distributed Identity Management..... 16
Objective..... 16
Background..... 16
Procedure..... 16
Results..... 18
Analysis..... 18
Conclusion..... 18
Lab 9: SUID and SGID Files..... 18
Objective..... 18
Background..... 18
Procedure..... 18
Results..... 19
Analysis..... 19
Conclusion..... 19
Lab 10: File Attributes..... 19
Objective..... 19
Background..... 19
Procedure..... 20
Results..... 20
Analysis..... 20
Conclusion..... 20
Lab 11: Shared Group Directory..... 21
Objective..... 21
Background..... 21
Procedure..... 21
Results..... 22
Analysis..... 22
Conclusion..... 22
Lab 12: SELinux Type Enforcement..... 22
Objective..... 22
Background..... 22
Procedure..... 22
Results..... 23
Analysis..... 23
Conclusion..... 23
Overall Conclusion: Strengthening the Linux Security Perimeter..... 24
Key Pillars of the Secured Environment..... 24
Final Assessment..... 24

Lab 1: Assigning Limited Sudo Privileges

Objective

The objective of this lab was to understand how to manage user privileges in Linux by assigning different levels of administrative access using the sudo mechanism.

Background

In Linux systems, administrative privileges are controlled using the sudo command. Instead of granting full root access, sudo allows fine-grained delegation of administrative tasks. This follows the **principle of least privilege**, where users are given only the permissions necessary to perform their duties.

Procedure

1. Created three users: Lionel, Katelyn, and Maggie.

```
[yunwen@localhost yunwen]# sudo useradd -m lionel
[yunwen@localhost yunwen]# sudo useradd -m katelyn
[yunwen@localhost yunwen]# sudo useradd -m maggie
```

2. Assigned passwords to each account.

```
[yunwen@localhost yunwen]# sudo passwd lionel
[yunwen@localhost yunwen]# sudo passwd katelyn
[yunwen@localhost yunwen]# sudo passwd maggie
```

3.
4. Opened the sudo configuration file using visudo.
5. Modified the configuration:
    - Lionel: Granted full administrative privileges.
    - Katelyn: Allowed only to run systemctl status sshd.
    - Maggie: Allowed to execute commands under the STORAGE alias.

```
# Define the STORAGE alias for Maggie
Cmnd_Alias STORAGE = /usr/sbin/fdisk, /usr/sbin/parted,
/usr/bin/lsblk

# User privilege specifications
lionel  ALL=(ALL:ALL) ALL
katelyn ALL=(ALL) /usr/bin/systemctl status sshd
maggie  ALL=(ALL) STORAGE
```

6. Switched into each user account using su and tested command execution.

Results and Observations

- Lionel successfully executed all administrative commands.
- Katelyn could only check the SSH service status but could not restart it.
- Maggie had restricted access limited to storage-related operations.
- Unauthorised commands resulted in permission denial errors.

Analysis

This lab demonstrated how sudo can restrict user actions at a granular level. It prevents misuse of system privileges while maintaining operational efficiency.

Conclusion

Fine-grained privilege delegation is essential in multi-user environments to enhance system security and accountability.

Lab 2: Disabling the Sudo Timer

Objective

To understand and modify sudo authentication timeout behavior.

Background

By default, sudo caches user authentication for a short period. This can be a security risk if a user leaves their session unattended.

Procedure

1. Executed several sudo commands to observe default behavior.

```
[yunwen@localhost yunwen]# sudo ls /root
[sudo] password for yunwen:
[yunwen@localhost yunwen]# sudo ls /root
anaconda-ks.cfg
[yunwen@localhost yunwen]# sudo -k
[yunwen@localhost yunwen]# sudo ls /root
[sudo] password for yunwen:
```

2. Used sudo -k to invalidate cached credentials.

```
[yunwen@localhost yunwen]# sudo visudo
```

3. Edited the sudoers file to set:

```
Defaults timestamp_timeout=0
```

4. Tested command execution again.

```
[yunwen@localhost yunwen]# sudo ls /root
[sudo] password for yunwen:
anaconda.cfg
[yunwen@localhost yunwen]# sudo ls /root
[sudo] password for yunwen:
```

5. Applied user-specific timeout for Lionel.

```
Defaults timestamp_timeout=5
Defaults:lionel timestamp_timeout=0
```

```
[yunwen@localhost yunwen]# su - lionel
[lionel@localhost ~]# sudo whoami
[sudo] password for lionel:
root
[lionel@localhost ~]# sudo whoami
[sudo] password for lionel:
root
```

Results

- With timeout set to 0, password was required for every command.
- User-specific configuration worked independently.

Analysis

Disabling the sudo timer enhances security in shared environments but may reduce usability.

Conclusion

Sudo timeout settings should be adjusted based on the security requirements of the system.

Lab 3: Encrypted Home Directory

Objective

To create and manage encrypted home directories.

Background

Encrypting user data ensures confidentiality, especially in case of unauthorized physical access.

Procedure

1. Installed ecryptfs-utils.

```
sudo apt update && sudo apt install ecryptfs-utils libpam-ecryptfs -y
```

2. Created user Cleopatra with encrypted home directory.

```
[yunwen@localhost yunwen]# sudo adduser --encrypt-home cleopatra
Adding user `cleopatra' ...
Adding new group `cleopatra' (1004) ...
Adding new user `cleopatra' (1004) with group `cleopatra' ...
Creating home directory `/home/cleopatra' ...
Copying files from `/etc/skel' ...
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully›
```

3. Logged into the account and retrieved encryption passphrase.

```
[yunwen@localhost yunwen]# su - cleopatra
[cleopatra@localhost ~]# ecryptfs-unwrap-passphrase
Passphrase:
```

```
[cleopatra@localhost ~]# mount | grep ecryptfs
/home/cleopatra/.Private on /home/cleopatra type ecryptfs
(rw,relatime,ecryptfs_sig=...,ecryptfs_cipher=aes,ecryptfs_key_bytes=16,...)

[cleopatra@localhost ~]# touch secret_file.txt
[cleopatra@localhost ~]# exit
```

```
[yunwen@localhost yunwen]# ls /home/cleopatra
[yunwen@localhost yunwen]# ls /home/cleopatra/.Private
ECRYPTFS_FNEK_ENCRYPTED.FWa1b2c3d4e5
ECRYPTFS_FNEK_ENCRYPTED.GXf9h8g7f6e5
```

Results

- Home directory was encrypted successfully.
- Data remained inaccessible without proper authentication.

Analysis

Encryption adds layer of security beyond file permissions.

Conclusion

Encrypted home directories are essential for protecting sensitive user data.

Lab 4: Password Complexity Criteria

Objective

To enforce strong password policies.

Background

Weak passwords are a major vulnerability in system security.

Procedure

1. Configured password policy file.

```
[yunwen@localhost yunwen]# sudo apt update && sudo
apt install libpam-pwquality -y
```

2. Set minimum password length.

```
[yunwen@localhost yunwen]# sudo nano
/etc/security/pwquality.conf
```

```
# Minimum length of 12 characters
minlen = 12

# Require at least one uppercase letter
ucredit = -1

# Require at least one lowercase letter
lcredit = -1

# Require at least one digit
dcredit = -1

# Require at least one special character (symbol)
ocredit = -1

# Maximum number of repeat characters
maxrepeat = 3
```

3. Tested weak and strong passwords.

```
[yunwen@localhost yunwen]# sudo passwd cleopatra
New password:
Retype new password:
BAD PASSWORD: The password is shorter than 12 characters
passwd: Authentication token manipulation error
```

4. Configured complexity rules using character classes.

```
[yunwen@localhost yunwen]# sudo passwd cleopatra
New password:
Retype new password:
BAD PASSWORD: The password fails the dictionary check - it does
not contain enough different characters
passwd: Authentication token manipulation error
```

```
[yunwen@localhost yunwen]# sudo passwd cleopatra
New password:
Retype new password:
passwd: password updated successfully
```

Results

- Weak passwords were rejected.
- Complex passwords were accepted.

Analysis

Password complexity rules significantly reduce brute-force attack success rates.

Conclusion

Strong password policies are critical for system security.

Lab 5: Account and Password Expiry

Objective

To manage user account lifecycle and password aging.

Background

Account expiration prevents unauthorized access from inactive accounts.

Procedure

1. Created user Samson with expiration date.

```
[yunwen@localhost yunwen]# sudo useradd -m -e 2026-12-31 samson
[yunwen@localhost yunwen]# sudo passwd samson
```

2. Modified expiration date.

```
[yunwen@localhost yunwen]# sudo chage -e 2027-01-15 samson
[yunwen@localhost yunwen]# sudo chage -l samson
Last password change          : Apr 17, 2026
Password expires          : never
Password inactive         : never
Account expires           : Jan 15, 2027
Minimum number of days between password change    : 0
Maximum number of days between password change    : 99999
Number of days of warning before password expires : 7
```

3. Forced password change on first login.

```
[yunwen@localhost yunwen]# sudo chage -d 0 samson
[yunwen@localhost yunwen]# su - samson
Password:
You are required to change your password immediately
(administrator enforced)
Current password:
New password:
Retype new password:
[samson@localhost ~]#
```

4. Configured password ageing policies.

```
[yunwen@localhost yunwen]# sudo chage -M 90 -W 10 samson
```

Results

- Expiry dates applied successfully.
- Password changes are enforced as configured.

Analysis

Proper account lifecycle management reduces long-term security risks.

Conclusion

Account and password expiration policies are essential in secure environments.

Lab 6: Detecting Compromised Passwords

Objective

To check passwords against known breach databases.

Background

Many users reuse passwords that may already be compromised.

Procedure

1. Queried the Pwned Passwords API.

```
[yunwen@localhost yunwen]# sudo apt update && sudo apt install
curl openssl -y
```

```
[yunwen@localhost yunwen]# nano check_pwned.sh
```

```
#!/bin/bash
PASSWORD=$1
# Generate SHA-1 hash and convert to uppercase
HASH=$(echo -n "$PASSWORD" | openssl sha1 | awk '{print
toupper($2)}')
PREFIX=${HASH:0:5}
SUFFIX=${HASH:5}

# Query the API with the prefix
RESULT=$(curl -s "https://api.pwnedpasswords.com/range/$PREFIX" |
grep -i "$SUFFIX")

if [ -z "$RESULT" ]; then
    echo "Success: No matches found. This password is safe."
else
    COUNT=$(echo "$RESULT" | cut -d: -f2)
    echo "CRITICAL: This password has been seen $COUNT times in
data breaches!"
fi
```

2. Executed script to test passwords.

```
[yunwen@localhost yunwen]# chmod +x check_pwned.sh
[yunwen@localhost yunwen]# ./check_pwned.sh password123
CRITICAL: This password has been seen 192843 times in data
breaches!
```

```
[yunwen@localhost yunwen]# ./check_pwned.sh L1nux_L4b_2026_!#
Success: No matches found. This password is safe.
```

Results

- Common passwords were flagged.
- Unique passwords showed no matches.

Analysis

Using breach databases helps identify insecure passwords proactively.

Conclusion

Password validation against breach databases improves overall system security.

Lab 7: Configuring pam_tally2

Objective

To implement account lockout after failed login attempts.

Background

Brute-force attacks rely on repeated login attempts.

Procedure

1. Modified PAM configuration files.

```
[yunwen@localhost yunwen]# sudo nano /etc/pam.d/common-auth
```

2. Set lockout threshold and unlock time.

```
auth required pam_tally2.so deny=3 unlock_time=600 onerr=fail
```

3. Simulated failed logins.

```
[yunwen@localhost yunwen]# su - samson
Password:
su: Authentication failure

[yunwen@localhost yunwen]# su - samson
Password:
su: Authentication failure

[yunwen@localhost yunwen]# su - samson
Password:
su: Authentication failure
# (The account is now locked)
```

4. Reset account manually.

```
[yunwen@localhost yunwen]# sudo pam_tally2 --user samson --reset
Login           Failures Latest failure     From
samson               3    04/17/26 22:40:15  pts/0

[yunwen@localhost yunwen]# sudo pam_tally2 --user samson
Login           Failures Latest failure     From
samson               0
# (The count is now zero; Samson can log in again)
```

Results

- Account locked after multiple failed attempts.
- Unlock required administrative intervention.

Analysis

Account lockout mechanisms effectively prevent brute-force attacks.

Conclusion

Login attempt restrictions are crucial for system protection.

Lab 8: Distributed Identity Management

Objective

To explore centralized identity management systems.

Background

Managing users across multiple systems individually is inefficient.

Procedure

1. Installed FreeIPA and related tools.

```
[yunwen@localhost yunwen]# sudo dnf install freeipa-server
freeipa-server-dns -y
[yunwen@localhost yunwen]# ipa-server-install --setup-dns
```

2. Administrative Authentication

Once installed, you must obtain a Kerberos Ticket to perform administrative actions. Without this ticket, the ipa commands will fail.

```
[yunwen@localhost yunwen]# kinit admin
Password for admin@IPA.LOCAL:
[yunwen@localhost yunwen]# klist
Ticket cache: KEYRING:persistent:0:0
Default principal: admin@IPA.LOCAL
Valid starting       Expires              Service principal
04/17/26 23:10:05    04/18/26 23:10:05
krbtgt/IPA.LOCAL@IPA.LOCAL
```

3. Configured centralized authentication.

```
[yunwen@localhost yunwen]# ipa user-add liam --first=Liam --
last=Smith --password
Password:
Enter Password again to verify:
------------------
Added user "liam"
------------------
  User login: liam
  First name: Liam
  Last name: Smith
  Full name: Liam Smith
  Home directory: /home/liam
  Login shell: /bin/sh
```

4. Client Enrollment

```
[yunwen@client-node ~]# sudo dnf install freeipa-client -y
[yunwen@client-node ~]# sudo ipa-client-install --mkhomedir
```

```
[yunwen@client-node ~]# ssh liam@localhost
Password:
WARNING: Your password has expired.
You must change your password now and login again!
Changing password for liam.
Current password:
New password:
Retype new password:
[liam@client-node ~]$ whoami
liam
[liam@client-node ~]$ pwd
/home/liam
```

To allow other machines to use this central database, you "enroll" them as clients.

Results

- Users could be managed centrally.

Analysis

Centralised systems improve scalability and security.

Conclusion

Identity management systems simplify administration in large environments.

Lab 9: SUID and SGID Files

Objective

To identify and analyze privileged files.

Background

SUID/SGID files can execute with elevated privileges.

Procedure

1. Scanned filesystem for such files.

```
[yunwen@localhost yunwen]# find /usr/bin -perm /6000 -type f
2>/dev/null
# (Output lists common binaries like /usr/bin/passwd and
/usr/bin/sudo)
```

2. Created test file with SUID.

```
[yunwen@localhost yunwen]# touch suid_test
[yunwen@localhost yunwen]# ls -l suid_test
-rw-r--r--. 1 root root 0 Apr 17 22:55 suid_test
[yunwen@localhost yunwen]# chmod 4755 suid_test
[yunwen@localhost yunwen]# ls -l suid_test
-rwsr-xr-x. 1 root root 0 Apr 17 22:55 suid_test
```

3. Compared results.

```
[yunwen@localhost yunwen]# find . -perm -4000 -type f
./suid_test
```

Results

- New SUID file detected.

Analysis

Unmonitored SUID files can be exploited.

Conclusion

Regular audits are necessary to detect security risks.

Lab 10: File Attributes

Objective

To explore advanced file protection mechanisms.

Background

Linux provides extended attributes for file control.

Procedure

1. Created test file.

```
[yunwen@localhost yunwen]# touch system.log
[yunwen@localhost yunwen]# echo "Initial Log Entry" > system.log
[yunwen@localhost yunwen]# lsattr system.log
```

2. Applied append-only and immutable attributes.

```
[yunwen@localhost yunwen]# sudo chattr +a system.log
[yunwen@localhost yunwen]# echo "Overwriting" > system.log
-bash: system.log: Operation not permitted

[yunwen@localhost yunwen]# echo "New Log Entry" >> system.log
[yunwen@localhost yunwen]# rm system.log
rm: cannot remove 'system.log': Operation not permitted
```

3. Attempted modifications.

```
[yunwen@localhost yunwen]# sudo chattr -a system.log
[yunwen@localhost yunwen]# sudo chattr +i system.log

[yunwen@localhost yunwen]# echo "Extra data" >> system.log
-bash: system.log: Operation not permitted

[yunwen@localhost yunwen]# mv system.log system.old
mv: cannot move 'system.log' to 'system.old': Operation not
permitted
```

Results

- Append-only allowed additions only.
- Immutable prevented all changes.

Analysis

These attributes provide strong protection against accidental or malicious changes.

Conclusion

File attributes are powerful tools for system integrity.

Lab 11: Shared Group Directory

Objective

To configure secure shared access.

Background

Group collaboration requires controlled permissions.

Procedure

1. Created group and users.

```
[yunwen@localhost yunwen]# sudo groupadd developers
[yunwen@localhost yunwen]# sudo usermod -aG developers lionel
[yunwen@localhost yunwen]# sudo usermod -aG developers katelyn
```

2. Configured shared directory.

```
[yunwen@localhost yunwen]# sudo mkdir /opt/project_alpha
[yunwen@localhost yunwen]# sudo chown :developers
/opt/project_alpha
[yunwen@localhost yunwen]# sudo chmod 2770 /opt/project_alpha
```

3. Applied ACLs.

```
[yunwen@localhost yunwen]# sudo setfacl -m u:maggie:rx
/opt/project_alpha
[yunwen@localhost yunwen]# getfacl /opt/project_alpha
# file: opt/project_alpha
# owner: root
# group: developers
# flags: s--
user::rwx
user:maggie:r-x
group::rwx
mask::rwx
other::---
```

4. Tested access.

```
[yunwen@localhost yunwen]# su - lionel
[lionel@localhost ~]$ touch /opt/project_alpha/code.py
[lionel@localhost ~]$ ls -l /opt/project_alpha/code.py
-rw-rw-r--. 1 lionel developers 0 Apr 17 23:05 code.py
```

```
[lionel@localhost ~]$ exit
[yunwen@localhost yunwen]# su - maggie
[maggie@localhost ~]$ ls /opt/project_alpha
code.py
[maggie@localhost ~]$ touch /opt/project_alpha/hack.txt
touch: cannot touch '/opt/project_alpha/hack.txt': Permission
denied
```

Results

- Users collaborated successfully.
- Unauthorized actions blocked.

Analysis

Combining permissions, SGID, sticky bit, and ACL ensures secure collaboration.

Conclusion

Proper permission management is essential in shared environments.

Lab 12: SELinux Type Enforcement

Objective

To understand SELinux security contexts.

Background

SELinux enforces mandatory access control policies.

Procedure

1. Installed Apache server.

```
[yunwen@localhost yunwen]# sudo dnf install httpd -y
[yunwen@localhost yunwen]# sudo systemctl start httpd
```

2. Created test webpage.

```
[yunwen@localhost yunwen]# echo "Welcome to SELinux Lab" >
/var/www/html/index.html
[yunwen@localhost yunwen]# ls -Z /var/www/html/index.html
unconfined_u:object_r:httpd_sys_content_t:s0
/var/www/html/index.html
```

3. Changed SELinux context incorrectly.

```
[yunwen@localhost yunwen]# touch ~/test.html
[yunwen@localhost yunwen]# mv ~/test.html /var/www/html/
[yunwen@localhost yunwen]# ls -Z /var/www/html/test.html
unconfined_u:object_r:admin_home_t:s0 /var/www/html/test.html
[yunwen@localhost yunwen]# curl http://localhost/test.html
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head><title>403 Forbidden</title></head>...
```

4. Restored correct context.

```
[yunwen@localhost yunwen]# sudo restorecon -v
/var/www/html/test.html
relabeled /var/www/html/test.html from
unconfined_u:object_r:admin_home_t:s0 to
unconfined_u:object_r:httpd_sys_content_t:s0
[yunwen@localhost yunwen]# curl -I http://localhost/test.html
HTTP/1.1 200 OK
```

Results

- Incorrect context caused access denial.
- Restoring context fixed the issue.

Analysis

SELinux strictly enforces access rules based on context.

Conclusion

SELinux significantly enhances system security by enforcing strict policies.

Overall Conclusion: Strengthening the Linux Security Perimeter

The series of labs conducted demonstrates that Linux security is not a single setting, but a **multi-layered architecture** designed to uphold the **Principle of Least Privilege**. By progressing from basic user management to advanced mandatory access controls, we have established a robust defence-in-depth strategy.

Key Pillars of the Secured Environment

- **Granular Privilege Management:** Through the use of sudo and the restriction of the root account, we ensured that users like Katelyn and Maggie have exactly the permissions required for their roles—and nothing more. Disabling the sudo timer further tightened this by eliminating "walk-away" vulnerabilities.
- **Data Confidentiality & Integrity:** Implementing **Encrypted Home Directories** protects data at rest, while advanced **File Attributes** (immutable and append-only) protect critical system logs and configurations from being tampered with, even by privileged accounts.
- **Identity & Access Control (IAM):** We moved from localised, weak security to enterprise-grade standards by enforcing **Password Complexity**, checking against **Breach Databases**, and implementing **Account Lockouts** via pam_tally2. The exploration of **FreeIPA** highlighted the efficiency and security of centralised identity management in scalable environments.
- **Proactive Auditing & Hardening:** Regular audits of **SUID/SGID** files and the enforcement of **SELinux Type Enforcement** ensure that the system remains resilient even if individual services are compromised. SELinux, in particular, proved to be a critical final line of defence against lateral movement.

Final Assessment

A secure Linux system is a moving target. The transition from **Discretionary Access Control (DAC),** where users control their own files, to **Mandatory Access Control (MAC),** where the system policy dictates safety, is essential for modern infrastructure.

Ultimately, these labs prove that while complexity can be an obstacle to usability, a well-configured system uses automation (like chage for expiry and restorecon for SELinux) to maintain a high security posture with minimal administrative overhead. Regular auditing and the consistent application of these protocols are the only ways to stay ahead of evolving threats.